

Ejercicios del Tema 1

Ejercicio 1

Diseñe usando UML la clase Edificio. Esta clase tiene un método privado (`ponerNumeroDePersonas` que se le pasa un número entero) y 6 métodos públicos (`obtenerNumeroDePersonas` que devuelve un entero, `entrarPersona`, `salirPersona`, `estaElEdificioAbierto` que devuelve un booleano, `abrirEdificio` y `cerrarEdificio`). Además, la clase define 2 propiedades privadas (`numeroDePersonas` de tipo entero y `edificioAbierto` de tipo booleano).

Ejercicio 2

La empresa EnlacesProfesionales desea crear un software para gestionar currículums de personas.

Cada curriculum está formado por 3 secciones: los datos personales, la formación académica y la experiencia profesional.

Entre los datos personales, al menos, debe almacenarse: el nombre, los apellidos y la dirección.

En la formación académica debe almacenarse los diferentes estudios que haya cursado una persona. De cada uno de ellos se debe recoger al menos: la titulación, los años en los que se estudió, la nota media obtenida.

En la sección de experiencia profesional debe almacenarse los diferentes trabajos en los que haya estado contratado. De cada uno de ellos se debe recoger: el nombre de la empresa, la fecha de inicio del trabajo y la fecha de fin.

Además, cada curriculum puede tener enlaces a los curriculums de otras personas que corroborarían tu currículum.

Diseñe, usando UML, un diagrama con las clases que considere necesario para representar estos datos.

Ejercicio 3

La empresa Electrical construye cajeros electrónicos para poner en las paredes de los bancos. Esta empresa vende un cajero que dispone de una pantalla, un teclado qwerty, una impresora, un lector de tarjetas y un dispensador de billetes. La empresa desea ofrecer una clase Java llamada `CajeroElectrical` que permita operar estos elementos creando un objeto de dicha clase. Las operaciones que debe realizar dicha clase son:

- Recoger tarjeta, que pone el cajero en un modo en el que se espera la introducción de una tarjeta. Este modo no termina hasta que no se introduce una tarjeta. En ese momento se devuelve el identificador de la tarjeta en forma de String.
- Expulsar tarjeta, que expulsa la última tarjeta recogida.
- Mostrar pantalla, indicando mediante una cadena de texto la información que se mostrará.
- Imprimir recibo, indicando mediante una cadena de texto la información que se imprimirá.
- Leer cadena desde el teclado.

- Consultar si hay suficiente dinero para realizar una operación. A esta operación hay que pasarle la cantidad que se desea sacar. Dicha cantidad deberá ser múltiplo de 10. Este método devolverá `false` o `true` para indicar si se puede entregar dicha cantidad. En caso de `false`, también devolverá los billetes que no se pueden utilizar (por ejemplo no quedan billetes de 20). Para ello se puede crear una clase auxiliar que contenga ambos elementos.
- Expulsar dinero, indicando la cantidad y el tiempo que el dinero estará disponible en la ranura de salida. Además, este método devolverá `false` o `true` para indicar si el cliente recogió el dinero.

Dibuje un diagrama UML de clases con la clase `CajeroElectrical`, la clase auxiliar y la relación que hay entre ambas.

Ejercicio 4

En el contexto del ejercicio anterior, supóngase que se dispone de una clase que permite la comunicación con la base de datos del banco de la Luna Amarilla. Dicha clase dispone de cinco métodos:

- `verifyPIN`, que recibe el identificador de la tarjeta y el PIN para comprobar si es correcto.
- `changePIN`, que permite informar al banco de un nuevo PIN elegido por el usuario.
- `testOperation`, que recibe el identificador de la tarjeta y el dinero que se desea retirar (negativo) o ingresar (positivo). Este método devuelve si la operación se puede realizar o en caso contrario el motivo del rechazo (superada la cantidad máxima permitida al día, falta de fondos...). También devuelve la comisión que aplicará el banco.
- `anotateOperation`, que recibe el identificador de la tarjeta y el dinero que se desea retirar (negativo) o ingresar (positivo). Este método devuelve si la operación se ha realizado correctamente o en caso contrario el motivo del rechazo (superada la cantidad máxima permitida al día, falta de fondos...). También devuelve la comisión que ha aplicado el banco.
- `lastMovements`, que permite consultar los últimos movimientos. Dicho método recibe el identificador de la tarjeta y devuelve un `String` con las últimas operaciones realizadas.

Dibuje el diagrama UML de dicha clase y las clases auxiliares que se consideren oportunas.

Ejercicio 5

Supóngase que el Banco de la Luna Amarilla desea instalar cajeros automáticos para que sus clientes puedan consultar el saldo, sacar dinero y cambiar el PIN mediante una tarjeta de débito.

En general la operativa consiste en que el cliente inserta la tarjeta en el cajero. Luego se le pide la contraseña. Si es correcta se le presenta un menú con las operaciones posibles. Cuando el usuario solicita una operación se ejecutan las acciones necesarias, de comunicación con el banco y de operación del cajero, para completarlas. Finalmente, se devuelve la tarjeta al cliente si no se desea realizar más operaciones.

En la operativa, el software que se desarrolle podrá utilizar las clases diseñadas en los ejercicios anteriores (3 y 4).

Usando el paradigma de orientación a objetos y el lenguaje UML se pide diseñar un sistema que resuelva el enunciado. Realice el diseño suponiendo que el banco desea que más tarde sea posible añadir nuevas operaciones al cajero como: comprar entradas de teatro, realizar ingresos, enviar dinero a otras cuentas, etc...

Ejercicio 6

Responde razonadamente a las siguientes preguntas:

- a) ¿Por qué las clases suelen tener nombres en singular?
- b) ¿Por que facilita la programación la posibilidad de definir abstracciones?
- c) ¿Por qué crees que las propiedades deben ser privadas?
- d) ¿Pon un ejemplo de rotura de la invariante de una clase?
- e) ¿Por qué se deben crear métodos privados?
- f) ¿Por qué se recomienda que los métodos tengan pocas líneas de código?